

L11 · Άπληστοι Αλγόριθμοι I

Χρονοπρογραμματισμός Διαστημάτων

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (K17) · ΕΚΠΑ

Η ιδέα του άπληστου αλγορίθμου, ο χρονοπρογραμματισμός διαστημάτων (interval scheduling) με το κριτήριο του χρόνου λήξης, η απόδειξη ορθότητας, και η διαμέριση διαστημάτων.

1 Τι θα δούμε

Ξεκινάει μια νέα οικογένεια αλγορίθμων: οι **άπληστοι** (greedy). Ήδη συναντήσαμε τρεις στο μάθημα για τα γραφήματα — Dijkstra, Prim, Kruskal — αλλά εκεί η απληστία ήταν «κρυμμένη» μέσα στο γράφημα. Εδώ τη βλέπουμε καθαρά, σε δύο προβλήματα χρονοπρογραμματισμού:

1. **Χρονοπρογραμματισμός διαστημάτων** — διάλεξε όσο το δυνατόν περισσότερες μη-επικαλυπτόμενες εργασίες.
2. **Διαμέριση διαστημάτων** — μοίρασε όλες τις εργασίες σε όσο το δυνατόν λιγότερες μηχανές.

Το πιο σημαντικό μάθημα εδώ **δεν** είναι ο αλγόριθμος — είναι το πώς **αποδεικνύεις** ότι ένας άπληστος αλγόριθμος είναι σωστός. Θα το χρειαστείς ξανά και ξανά στην εξεταστική.

2 Η ιδέα του άπληστου αλγορίθμου

Κλειδί

Άπληστος αλγόριθμος: χτίζει τη λύση **σταδιακά**, και σε κάθε βήμα παίρνει την απόφαση που φαίνεται **τοπικά καλύτερη** — χωρίς να ξανασκέφτεται, χωρίς να κοιτάει το μέλλον.

Διαίσθηση

Διαισθητικά: ο άπληστος αλγόριθμος είναι σαν να ανεβαίνεις βουνό κοιτάζοντας μόνο τα πόδια σου — κάθε βήμα προς τα πάνω. Μερικές φορές αυτό σε φέρνει στην κορυφή· άλλες φορές σε παγιδεύει σε έναν μικρό λοφίσκο. **Το ζητούμενο σε κάθε greedy πρόβλημα είναι το ίδιο: ποιο τοπικό κριτήριο εγγυάται την παγκόσμια βέλτιστη λύση — και γιατί;**

Δεν αρκεί να βρεις έναν άπληστο κανόνα που «δείχνει λογικός». Πρέπει να τον **αποδείξεις**. Όπως θα δούμε αμέσως, οι περισσότεροι λογικοφανείς κανόνες είναι **λάθος**.

3 Χρονοπρογραμματισμός Διαστημάτων (Interval Scheduling)

3.1 Διατύπωση

Είσοδος: n εργασίες. Η εργασία j ξεκινά τη χρονική στιγμή s_j και τελειώνει τη στιγμή f_j .

Συμβατές εργασίες: δύο εργασίες είναι **συμβατές** αν δεν επικαλύπτονται χρονικά.

Στόχος: βρες το **μεγαλύτερο σε πλήθος** υποσύνολο εργασιών που είναι ανά δύο συμβατές.

Διαίσθηση

Πραγματικό παράδειγμα: μία αίθουσα, πολλές αιτήσεις για κρατήσεις, καθεμία με ώρα έναρξης και λήξης. Θέλεις να ικανοποιήσεις **όσο το δυνατόν περισσότερες** — όχι τις «μεγαλύτερες» ή τις «σημαντικότερες», απλώς το μέγιστο πλήθος.

3.2 Το άπληστο πρότυπο

Όλοι οι υποψήφιοι αλγόριθμοι έχουν την ίδια σκελετική μορφή:

Θεώρησε τις εργασίες σε **κάποια σειρά**. Διάλεξε κάθε εργασία εφόσον είναι συμβατή με όσες έχεις ήδη επιλέξει.

Όλο το παιχνίδι είναι **ποια σειρά**. Τέσσερις λογικοφανείς υποψήφιες:

Κριτήριο	Σειρά
Μικρότερος χρόνος έναρξης	αύξουσα ως προς s_j
Μικρότερος χρόνος λήξης	αύξουσα ως προς f_j
Μικρότερο διάστημα	αύξουσα ως προς $f_j - s_j$
Ελάχιστες διενέξεις	αύξουσα ως προς c_j (πλήθος μη-συμβατών εργασιών)

3.3 Τρία κριτήρια που αποτυγχάνουν

Προσοχή

Μικρότερος χρόνος έναρξης — ΛΑΘΟΣ. Η εργασία που ξεκινάει πρώτη μπορεί να είναι τεράστια και να επικαλύπτει πολλές άλλες, μπλοκάροντας μια λύση με περισσότερες εργασίες.

Μικρότερο διάστημα — ΛΑΘΟΣ. Η συντομότερη εργασία μπορεί να «κάθεται» ακριβώς ανάμεσα σε δύο εργασίες που μεταξύ τους είναι συμβατές — διαλέγοντάς την, χάνεις και τις δύο.

Ελάχιστες διενέξεις — ΛΑΘΟΣ. Με ανάλογο αντιπαράδειγμα, η εργασία με τις λιγότερες συγκρούσεις μπορεί και πάλι να μπλοκάρει μια μεγαλύτερη λύση.

Αντιπαράδειγμα για το «μικρότερο διάστημα». Έστω τρεις εργασίες: η A (διάστημα 5) και η C (διάστημα 4) είναι μεταξύ τους συμβατές, ενώ η B (διάστημα 2) κάθεται ανάμεσά τους και συγκρούεται και με τις δύο. Το κριτήριο «μικρότερο διάστημα» διαλέγει πρώτο το B — και τότε ούτε το A ούτε το C είναι πια συμβατά. Λύση: 1. Η βέλτιστη λύση $\{A, C\}$ έχει μέγεθος 2.

3.4 Το κριτήριο που δουλεύει: μικρότερος χρόνος λήξης

Κλειδί

Απληστος αλγόριθμος (earliest finish time). Θεώρησε τις εργασίες σε **αύξουσα σειρά χρόνου λήξης**. Διάλεξε κάθε εργασία εφόσον είναι συμβατή με τις ήδη επιλεγμένες.

Διαίσθηση

Γιατί ο χρόνος λήξης; Επειδή η εργασία που τελειώνει **νωρίτερα** αφήνει **όσο το δυνατόν περισσότερο ελεύθερο χρόνο** για όλες τις επόμενες. Είναι η πιο «γενναιόδωρη» επιλογή προς το μέλλον — και αυτή ακριβώς η διαίσθηση γίνεται η απόδειξη παρακάτω.

Χρονοπρογραμματισμός Διαστημάτων(s, f):

Ταξινόμηση των εργασιών ώστε $f_1 \leq f_2 \leq \dots \leq f_n$

$A \leftarrow \emptyset$

Για $j = 1$ έως n :

Εάν η εργασία j είναι συμβατή με το A :

$A \leftarrow A \cup \{j\}$

επίστρεψε A

Υλοποίηση σε $O(n \log n)$: η ταξινόμηση κοστίζει $O(n \log n)$. Για τον έλεγχο συμβατότητας δεν χρειάζεται να κοιτάμε όλο το A — αρκεί να θυμόμαστε την **τελευταία** εργασία j^* που προστέθηκε. Η εργασία j είναι συμβατή με το A **αν και μόνο αν** $s_j \geq f_{j^*}$. Άρα κάθε έλεγχος είναι $O(1)$.

(Σε ένα τυπικό στιγμιότυπο 8 εργασιών, ο άπληστος κατά χρόνο λήξης μπορεί να επιλέξει π.χ. το σύνολο $\{1, 4, 8\}$ μεγέθους 3 — που είναι το βέλτιστο.)

3.5 Ορθότητα — «ο άπληστος προηγείται»

Κλειδί

Θεώρημα. Ο άπληστος αλγόριθμος ως προς τον χρόνο λήξης επιστρέφει λύση **μέγιστου** πλήθους.

Απόδειξη (εις άτοπον). Έστω $i_1, i_2, \dots, i_k$ οι εργασίες που επιλέγει ο άπληστος, σε σειρά. Έστω $j_1, j_2, \dots, j_m$ μια **βέλτιστη** λύση, διαλεγμένη έτσι ώστε να **συμφωνεί με τον άπληστο για όσο το δυνατόν περισσότερες αρχικές εργασίες** — δηλαδή $i_1 = j_1, \dots, i_r = j_r$ για τη μέγιστη δυνατή τιμή του r . Αν ήταν $k < m$, ο άπληστος δεν είναι βέλτιστος. Θα φτάσουμε σε άτοπο.

Ο άπληστος «προηγείται». Η εργασία i_{r+1} που διάλεξε ο άπληστος μετά την i_r έχει χρόνο λήξης $f(i_{r+1}) \leq f(j_{r+1})$. Γιατί; Επειδή $i_r = j_r$, η εργασία j_{r+1} είναι συμβατή με την i_r — άρα ήταν **υποψήφια** όταν ο άπληστος διάλεγε την i_{r+1} . Και ο άπληστος επιλέγει πάντα την υποψήφια με τον **μικρότερο** χρόνο λήξης. Άρα $f(i_{r+1}) \leq f(j_{r+1})$.

Η ανταλλαγή. Φτιάχνουμε μια νέα λύση αντικαθιστώντας στη βέλτιστη την j_{r+1} με την i_{r+1} :

$$S' = j_1, \dots, j_r, i_{r+1}, j_{r+2}, \dots, j_m.$$

Είναι έγκυρη: η i_{r+1} είναι συμβατή με τις $j_1, \dots, j_r$ ($= i_1, \dots, i_r$) αφού ο άπληστος την επέλεξε, και επειδή $f(i_{r+1}) \leq f(j_{r+1})$ τελειώνει **όχι αργότερα** από την j_{r+1} , άρα δεν συγκρούεται ούτε με την j_{r+2} . Το S' έχει m εργασίες — άρα είναι **κι αυτό βέλτιστο** — αλλά συμφωνεί με τον άπληστο για $r+1$ εργασίες. **Αντίφαση** στη μεγιστότητα του r .

Μένει η περίπτωση $k = r < m$: τότε υπάρχει η j_{r+1} , συμβατή με την i_r , και ο άπληστος **δεν** θα είχε σταματήσει — θα την είχε εξετάσει και θα είχε προσθέσει κάτι. Άτοπο. Άρα $k = m$. ■

Διαίσθηση

Το όνομα της τεχνικής: «ο άπληστος προηγείται» (greedy stays ahead). Δείχνεις ότι μετά από κάθε βήμα, η μερική λύση του άπληστου είναι **τουλάχιστον τόσο καλή** όσο οποιασδήποτε άλλης λύσης — εδώ, ότι τελειώνει νωρίτερα. Θα δούμε άλλες δύο τεχνικές απόδειξης στο επόμενο μάθημα.

4 Διαμέριση Διαστημάτων (Interval Partitioning)

4.1 Διατύπωση

Αλλάζουμε το ερώτημα. Τώρα **πρέπει** να προγραμματίσουμε **όλες** τις εργασίες.

Είσοδος: n εργασίες, η j με (s_j, f_j) .

Στόχος: βρες το **ελάχιστο πλήθος μηχανών** ώστε να αναθέσεις κάθε εργασία σε μία μηχανή, με δύο εργασίες που τρέχουν ταυτόχρονα **ποτέ** στην ίδια μηχανή.

Διαίσθηση

Πραγματικό παράδειγμα: μαθήματα που πρέπει όλα να γίνουν, και ψάχνεις τον **ελάχιστο αριθμό αιθουσών**. Δύο μαθήματα που πέφτουν την ίδια ώρα χρειάζονται διαφορετική αίθουσα.

4.2 Το κάτω φράγμα: το βάθος

Κλειδί

Βάθος ενός συνόλου διαστημάτων: το **μέγιστο πλήθος** διαστημάτων που περιέχουν μια **κοινή** χρονική στιγμή.

Αν σε κάποια στιγμή τρέχουν ταυτόχρονα d εργασίες, αυτές χρειάζονται d διαφορετικές μηχανές. Άρα:

$$\text{πλήθος μηχανών} \geq \text{βάθος.}$$

Το ερώτημα είναι: **μπορούμε πάντα να πετύχουμε ακριβώς το βάθος;**

4.3 Ο άπληστος αλγόριθμος

Κλειδί

Άπληστο κριτήριο. Θεώρησε τις εργασίες σε **αύξουσα σειρά χρόνου έναρξης**. Ανάθεσε κάθε εργασία σε μια **οποιαδήποτε ελεύθερη μηχανή**· αν δεν υπάρχει, **άνοιξε** μια νέα.

Διαμέριση Διαστημάτων(s, f):

```
Ταξινομήσε τις εργασίες ώστε  $s_1 \leq s_2 \leq \dots \leq s_n$   
 $d \leftarrow 0$  // πλήθος μηχανών
```

```
Για  $j = 1$  έως  $n$ :
```

```
    Εάν η εργασία  $j$  είναι συμβατή με κάποια ήδη ανοιγμένη μηχανή  $k$ :  
        ανάθεσε την  $j$  στη μηχανή  $k$ 
```

```
    Αλλιώς:
```

```
         $d \leftarrow d + 1$ 
```

άνοιξε τη μηχανή d και ανάθεσε την j σε αυτήν
επίστρεψε d

4.4 Ορθότητα

Παρατήρηση πρώτα: ο αλγόριθμος δεν αναθέτει **ποτέ** δύο συγκρουόμενες εργασίες στην ίδια μηχανή — εξ ορισμού αναθέτει σε μηχανή μόνο αν είναι «συμβατή» με ό,τι έχει ήδη εκεί. Άρα η λύση είναι **έγκυρη**. Μένει να δείξουμε ότι χρησιμοποιεί **ελάχιστες** μηχανές.

Κλειδί

Θεώρημα. Ο άπληστος αλγόριθμος είναι βέλτιστος: χρησιμοποιεί ακριβώς $d =$ βάθος μηχανές.

Απόδειξη. Έστω d το πλήθος των μηχανών που άνοιξε ο αλγόριθμος. Η τελευταία μηχανή, η d -οστή, άνοιξε επειδή κάποια εργασία — έστω j — **δεν** ήταν συμβατή με καμία από τις $d - 1$ μηχανές που ήδη υπήρχαν.

Άρα, τη στιγμή που εξετάστηκε η j , καθεμία από τις $d - 1$ μηχανές είχε **τουλάχιστον μία** εργασία σε σύγκρουση με την j . Επειδή ο αλγόριθμος εξετάζει τις εργασίες σε **αύξουσα σειρά έναρξης**, όλες αυτές οι εργασίες ξεκίνησαν **πριν** την s_j — και αφού συγκρούονται με την j , δεν έχουν τελειώσει μέχρι την s_j .

Συνεπώς, τη χρονική στιγμή $s_j + \epsilon$ τρέχουν ταυτόχρονα d **εργασίες** (οι $d - 1$ συν την j). Άρα το βάθος είναι $\geq d$. Αλλά κάθε λύση χρειάζεται $\geq$ βάθος $\geq d$ μηχανές. Ο άπληστος χρησιμοποιεί ακριβώς d . Άρα είναι βέλτιστος. ■

Διαίσθηση

Το όνομα της τεχνικής: δομική εξήγηση (structural bound). Βρήκαμε ένα «εμπόδιο» μέσα στο ίδιο το πρόβλημα — το βάθος — που κανένας αλγόριθμος δεν μπορεί να ξεπεράσει, και δείξαμε ότι ο άπληστος ακριβώς αυτό το εμπόδιο πετυχαίνει. Όταν το κάτω φράγμα συναντά την τιμή του αλγορίθμου, ησυχάζεις.

Ανακεφαλαίωση

Τι κρατάμε από αυτή τη διάλεξη

1. **Άπληστος αλγόριθμος** — χτίζει λύση σταδιακά, κάθε βήμα τοπικά βέλτιστο. Το δύσκολο δεν είναι ο κανόνας, είναι η **απόδειξη**.
2. **Χρονοπρογραμματισμός διαστημάτων** — μέγιστο πλήθος συμβατών εργασιών. Σωστό κριτήριο: **μικρότερος χρόνος λήξης**. $O(n \log n)$.
3. Τα κριτήρια «μικρότερη έναρξη», «μικρότερο διάστημα», «ελάχιστες διενέξεις» **αποτυγχάνουν** — πάντα ζητά αντιπαράδειγμα πριν πιστέψεις έναν greedy κανόνα.
4. **Greedy stays ahead** — απόδειξη ότι η μερική λύση του άπληστου προηγείται κάθε άλλης σε κάθε βήμα.
5. **Διαμέριση διαστημάτων** — ελάχιστο πλήθος μηχανών. Κριτήριο: **μικρότερος χρόνος έναρξης**. Το αποτέλεσμα ισούται πάντα με το **βάθος**.
6. **Structural bound** — βρες ένα κάτω φράγμα μέσα στο πρόβλημα (το βάθος) και δείξε ότι ο αλγόριθμος το πετυχαίνει.